

CS473/573

Duration: 30 Minutes

Q2. (45 points)

You are given two lists: P contains weights of n people, S contains sizes of m T-shirts. You are also given an integer t that indicates how much a T-shirt can stretch. Each T-shirt is stretchy and a Person j in P can get a T-shirt i if and only if $S[i] - t \leq P[j] \leq S[i] + t$. In other words, a person can wear a T-shirt if his/her weight is inside the T-shirt's size interval. You are asked to maximize number of people getting a T-shirt. A person can get only one T-shirt and a T-shirt can be given to only one person.

Example: Consider two lists: $P=[70,50,40,80,84]$, $S=[32,91,45,20,33]$, $t=10$, and maximum number of T-shirts that can be distributed is 3:

Person with weight 40 gets the t-shirt with size 32. $(32-10 \leq 40 \leq 32+10)$

Person with weight 50 gets the t-shirt with size 45. $(45-10 \leq 50 \leq 45+10)$

Person with weight 84 gets the t-shirt with size 91. $(91-10 \leq 84 \leq 91+10)$

Note that the T-shirt with size 20 cannot be taken by anyone since no person is inside its size interval. Furthermore, T-shirts with size 32 and 33 can only be taken by the person with weight 40. Therefore, only one of them is given to the person with weight 40.

Q2a. (10 points)

Clearly indicate the **greedy choice** property in a single sentence. You do not need to give the proof of your choice.

Q2b. (35 points)

Provide a **greedy algorithm** in pseudocode that computes and returns the maximum number of T-shirts that can be distributed. Note that you are not required to return the list of T-shirts or people.

CHEAT SHEET

Greedy Algorithms:

Greedy Choice Property

- A greedy choice at each step yields a globally optimal solution.
- How to prove the greedy choice property?
 - o Consider the greedy choice c .
 - o Assume that there is an optimal solution B that doesn't contain c .
 - o Show that it is possible to convert B to another optimal solution B' , where B' contains c .

Optimal Substructure

A problem exhibits optimal substructure if an optimal solution to the problem contains optimal solutions to subproblems within it.

BILKENT UNIVERSITY
CS473 / 573 - Midweek Exam 5

Duration: 40 minutes

cs473atbilkent@gmail.com, UD: 533 364 9456, EB101: 552 289 9530, EB102: 555 418 9178, EB103: 537 519 6542

Q2. (50 pts)

There are n houses located on a straight line. The positions of the houses from left to right are given in a sorted array A . A cluster is defined as a set of at least four consecutive houses such that the distance between the leftmost house and the rightmost house is at most k . Your task is to group the houses into disjoint clusters such that the total number of clusters is maximized. It is allowed to leave some of the houses as unclustered.

Example. Suppose $A = [2, 4, 5, 7, 8, 10, 12, 19, 20, 24, 27, 30, 32, 35]$ and $k = 10$. Two possible clusterings are as follows:

- $\{\{2, 4, 5, 7, 8, 10, 12\}, \{20, 24, 27, 30\}\}$. 19, 32 and 35 are not included in any cluster. Including 19, 32 or 35 to either cluster would violate the distance constraint. There are two clusters but it is not the maximum number of clusters that can be achieved.
- $\{\{2, 4, 5, 7, 8\}, \{10, 12, 19, 20\}, \{24, 27, 30, 32\}\}$. 35 is not included in any cluster. There are three clusters and it is an optimal solution. Note that there can be multiple solutions.

Q2a. (15 Points) Clearly indicate the **greedy choice** property. You do not need to give the proof of your choice.

Q2b. (35 Points) Provide a **greedy algorithm** ($\text{MAX-CLUSTERING}(A, k)$) in pseudocode that finds and returns the set of disjoint clusters such that the total number of clusters is maximum among all possible clustering choices. If there are multiple solutions, you may return any one of them. Make sure to analyze the running time of your algorithm.

Q2a.

Let $S(A, i, k)$ denote an optimal solution of a subproblem that takes $A[i \dots n]$ as house location input.

$$S(A, i, k) = \begin{cases} \{\} & \text{if } i > n-3 \\ \{A[i], A[i+1], A[i+2], A[i+3]\} \cup S(A, i+4, k) & \text{if } A[i+3] - A[i] \leq k \\ S(A, i+1, k) & \text{otherwise} \end{cases}$$

If $A[4] - A[1] \leq k$, $\{A[1], A[2], A[3], A[4]\}$ is included in an optimal solution. Otherwise $A[1]$ belongs to no

Q2b.

MAX-CLUSTERING(A, k):

$C \leftarrow \{\}$ # C denotes the set of clusters $\Theta(1)$

$i \leftarrow 1$ $\Theta(1)$

while $i \leq n-3$ do:

if $A[i+3] - A[i] \leq k$ then:

$C \leftarrow C \cup \{A[i], A[i+1], A[i+2], A[i+3]\}$

$i \leftarrow i + 4$

else:

$i \leftarrow i + 1$

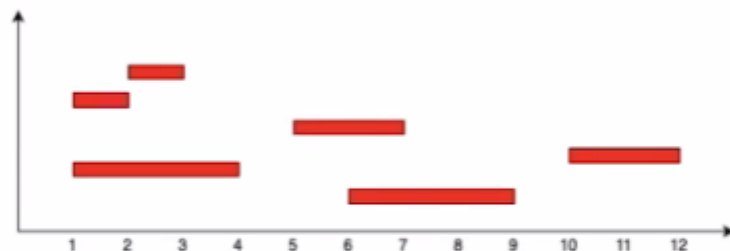
return C

• The running time of the algorithm is $\Theta(n)$.

Q2 (50 Points)

In a 2D plane, you have n targets to shoot. That is, each target has a positive y -coordinate and is located between a pair of x -coordinates $[x_1, x_2]$. Furthermore, each target has a distinct y -coordinate (no two targets are located in the same y -coordinate). Your aim is to shoot down all the targets with minimum number of bullets. For that, you will use your special bullets that do not stop after hitting a target and travel infinitely. You can only fire your gun from a specific line parallel to the x -axis, moving in a direction parallel to the y -axis. The input target list format is $\{[s_1, e_1], [s_2, e_2], \dots, [s_i, e_i], \dots, [s_n, e_n]\}$. Each coordinate pair $[s_i, e_i]$ corresponds to i th target's start and end x -coordinates, respectively.

Example: Consider the target list $\{[10, 12], [1, 2], [5, 7], [1, 4], [2, 3], [6, 9]\}$ shown in the diagram below:



You can shoot all the targets with three bullets:

- Shoot bullet 1 from $x = 2$. Bullet 1 will shoot down targets $[1, 2]$, $[1, 4]$ and $[2, 3]$.
- Shoot bullet 2 from anywhere between $x = [6, 7]$. Bullet 2 will shoot down targets $[5, 7]$ and $[6, 9]$.
- Shoot bullet 3 from anywhere between $x = [10, 12]$. Bullet 3 will shoot down target $[10, 12]$.

Q2a. (15 points) Clearly indicate the **greedy choice** property in a single sentence. You do not need to give the proof of your choice.

Q2b. (35 points) Provide a **greedy algorithm** in pseudocode that computes and returns the minimum number of bullets required to shoot down all the targets, making sure to analyze its runtime.

Unshot targets.
b) Let T be a list of coordinate pairs. Assume you can access start and end coordinates of i th target by using $T[i].start$ and $T[i].end$ respectively

Shoot-Targets(T, n)

$O(n \log n)$ { sort T in ascending order using $T[i].end$ (end coordinate of targets) as sorting key

tot-bullets = 1

cur-bullet = $T[1].start$ // assume T is 1 indexed

$O(n)$ { for $i \leftarrow 2$ to n
if cur-bullet $< T[i].start$
cur-bullet = $T[i].end$
tot-bullets++

return tot-bullets

total runtime $\Rightarrow O(n \log n)$

Bilkent University, CS 473/573, Session 2, 30pts, 40min

Q2: A student enters an exam with n questions. The i^{th} question is worth p_i points and the student can solve it in m_i minutes. There are no partial credits, and the student must spend m_i minutes to get full credit or leave the question blank. Duration of the exam is M minutes.

Describe, in **pseudocode**, an efficient algorithm which returns the questions to be solved in order to let the student earn the maximum grade in the exam. Assume that p_i, m_i and M are all integers, and the time it takes for the student to calculate and use the question set that yields an optimal grade can be ignored and will not count towards the duration of the exam.

Unlike other questions, you may **NOT** reuse any of the algorithms discussed in lectures in your solutions as a black box for this question.

n questions $\rightarrow p_i$ point
 m_i minute

duration M

maximize P total points

0-1 Knapsack

Define $c_{i,n}$ as the points of the solution for questions $1, \dots, i$ and max minute n .

$$c_{i,n} = \begin{cases} 0 & \text{if } i=0 \text{ or } n=0 \\ c_{i-1,n} & \text{if } m_i > n \\ \max(p_i + c_{i-1,n-m_i}, c_{i-1,n}) & \text{if } i > 0 \text{ and } n \geq m_i \end{cases}$$

QUESTION-SELECTOR(p, m, n, M)

let $c[0..n, 0..M]$ be a new array

for $n=0$ to M
 $c[0,n] = 0$

for $i=0$ to n
 $c[i,0] = 0$

for $i=1$ to n

 for $m=1$ to M

 if $m_i > m$
 $c[i,m] = c[i-1,m]$

 else $\max(p_i + c[i-1,m-m_i], c[i-1,m])$

$c[i,m] =$

return $c[n, M]$

Bilkent University, CS 473/573, Session 1, 35pts, 40min

Q1: You are given an array A of n positive integers. A subset S of the integers in A is called **T-distance subset** if the minimum difference between each pair of integers in S is larger or equal to T . Describe, in **pseudocode**, an **efficient greedy** algorithm that finds and returns a maximum sized T-distance subset for a given array A and distance T .

As an example, consider $A = \{10, 1, 1, 0, 25, 12, 9\}$ and $T = 3$. $B = \{10, 3, 7, 25\}$ and $C = \{3, 25, 12\}$ are both 3-distance subsets of A . However C is not maximum sized, since $|C| = 3$ and $|B| = 4$. Also notice that $D = \{10, 3, 7, 9, 25\}$ is not a 3-distance subset of A , since $9 - 7 < 3$.

Sort $A \rightarrow A = \{3, 5, 7, 9, 10, 12, 25\} = B$

greedy choice: select the smallest, consider the subset starting from the next available element.

T-DISTANCE-SUBSET(A, T)

$B = \text{SORT}(A)$

$C = \text{NEW LIST}()$

for $i = 0$ to $\text{LENGTH}(A)$:

if $\text{LENGTH}(C) == 0$ OR $A[i] \geq C[-1] + T$:

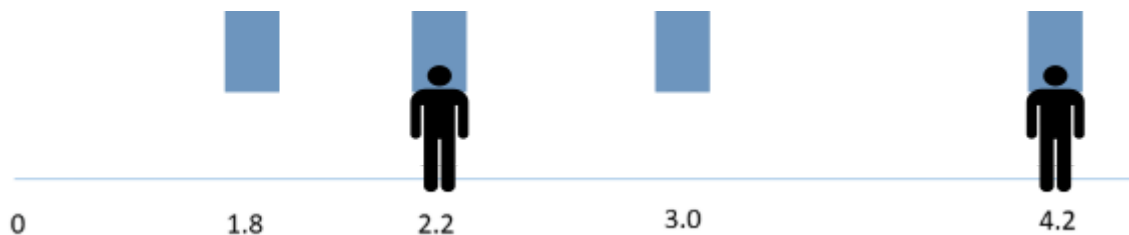
$C.append(A[i])$

return C

Bilkent University, CS 473/573, Session 3, 30pts, 30min

Question 3: In the art gallery guarding problem, you are given a set of real numbers in sorted order, that represent locations where paintings are hung in the straight hallway. Suppose that a single guard can protect all the paintings within distance at most 1 meter of his or her position (on both sides), and each guard must be positioned at the same location with a painting. Your job is to determine the minimum number of guards (as well as their locations) required to protect all the paintings.

For the example given below, there are 4 paintings with locations $\langle 1.8, 2.2, 3.0, 4.2 \rangle$. An optimal solution for this example is 2 guards and their locations are 2.2 and 4.2. Note that the first three paintings are protected by the first guard, whereas the last painting is protected by the second guard.



For each of the following greedy algorithms described below, either **prove the correctness** of the *greedy choice property* or **give a counter-example with at most four paintings** to show that it fails.

Clearly indicate your choice. If you provide a counterexample, give the locations of paintings, list of the guards for the optimal solution as well as the ones found by the algorithm.

- (a) Algorithm A: Mark each painting as unprotected. While there exist unprotected paintings, place a guard to **the leftmost unprotected painting**, mark paintings (as protected) that are protected by the new guard. Repeat.
- (b) Algorithm B: Mark each painting as unprotected. While there exist unprotected paintings, place a guard to the **next painting in the range of 1 meter of the leftmost unprotected painting**. If there is no such painting, place a guard to **the leftmost unprotected painting**. Mark paintings (as protected) that are protected by the new guard. Repeat.

BILKENT UNIVERSITY
Computer Engineering Department
CS 473/573 - Algorithms I
Midweek Exam 7

December 23, 2015

Surname: *Sert*
Name: *Uluç*

100

Duration: 100 minutes

Section: 2
Student ID: 21201238

1. (50 pts) Consider the problem of summing n positive floating point numbers, $x_1, x_2, \dots, x_n$ in such a way that the *rounding error* is minimized. Depending on how the summation is done, the number of summations that numbers participate in may differ. For example, x_1, x_2, x_3 , and x_5 will participate in 3 summations, while x_4 will participate in 1 summation in the following summation order:

$$(((x_1 + x_3) + (x_2 + x_5)) + x_4)$$

Note that both the order of the numbers and the placement of parentheses are chosen freely, however, each summation includes only two numbers, that is, you cannot sum three or more numbers at once. How large the rounding error of a sum of two floating point numbers is depends on how large the sum itself is (that is, a larger sum gives a larger error). Let p_i be the "parenthesis-depth" of floating point number x_i (that is, the number of parentheses surrounding the number). Then, the *rounding error* is defined as follows:

$$p_1 x_1 + p_2 x_2 + \dots + p_n x_n.$$

For example, consider the numbers $x_1 = 4.3$, $x_2 = 2.8$, $x_3 = 1.7$, $x_4 = 5.9$, and $x_5 = 3.4$. The summation order $((x_2 + x_5) + x_1) + (x_4 + x_3)$ gives the following rounding error:

$$2x_1 + 3x_2 + 2x_3 + 2x_4 + 3x_5 = 8.6 + 8.4 + 3.4 + 11.8 + 10.2 = 42.4.$$

Describe a greedy algorithm in pseudo-code which computes the summation order of n given positive floating point numbers with minimum rounding error.

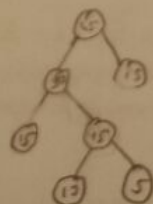
1)

(Note: Note means binary tree nodes)

Minimize Error (N) $I = |N|$ T = collection of nodes which are keyed to each N element Q = min-priority queue populated from T for $\ell-1$ times k = new node $k.\text{left} = \text{ExtractMin}(Q)$ $k.\text{right} = \text{ExtractMin}(Q)$ $k.\text{key} = k.\text{left}.\text{key} + k.\text{right}.\text{key}$ $\text{Insert}(Q, k)$ ✓return $\text{ExtractMin}(Q)$

Remark: The question is almost same as creating variable length code. Both, when expressed as a binary tree, has the cost of $\sum_{\text{leaves}} \text{depth} \times \text{value}$ and both minimize this cost. Thus Huffman's greedy algorithm fits for this problem. (Book page 431)

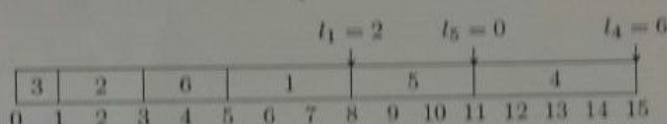
The returned solution is a binary tree containing the order in which the error is minimum. Example: ✓

means $((5+10+5)) + 15$

2. (50 pts) Suppose we are given jobs with deadlines and processing times to be scheduled on a single resource. If a job i starts at time s_i then it will finish at time $f_i = s_i + t_i$, where t_i is its processing time, and d_i is its deadline. The lateness of a job is $l_i = \max(0, f_i - d_i)$. We would like to schedule all jobs such that $L = \max l_i$ is minimized. For instance, given the following list of jobs along with their processing times and deadlines

i	1	2	3	4	5	6
t_i	3	2	1	4	3	2
d_i	6	8	9	9	14	15

following schedule yields $L = 6 = \max(0, 0, 0, 2, 0, 6)$, which is not optimal.



Give a **greedy algorithm** to determine a schedule to minimize L . Provide its running time complexity. Clearly indicate your greedy choice property.

Schedule (d, h)

$R \leftarrow []$

BUILD-MIN-HEAP (d, h)

for $i \leftarrow 1$ to h

$R[i] = \text{EXTRACT-MIN}(d, h-i)$

return R

you could just sort it.

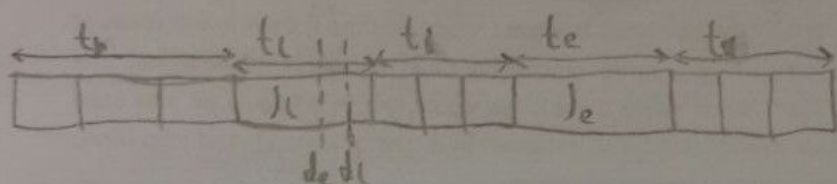
Running time complexity:

BUILD-MIN-HEAP is $\Theta(n)$. EXTRACT-MIN is $\Theta(\log n)$.

$$\Theta(1) + \Theta(n) + n \cdot \Theta(\log n) = \Theta(n \log n)$$

Greedy choice property: Scheduling the job with the closest deadline first will minimize the maximum lateness of the job, because if we consider a different (possibly not optimal) solution if the start time of two jobs are switched

than the lateness of the job with the later deadline.
cannot possibly increase to an amount larger than the original lateness of the job with the earlier deadline.



j_l : The job with the later deadline.

j_e : The job with the earlier deadline.

t_b : Total time taken by jobs before either j_l or j_e

t_i : Total time taken by jobs in between j_l and j_e .

t_a : Total time taken by jobs after j_l and j_e .

t_l : Time taken by j_l

t_e : Time taken by j_e .

If j_l comes before j_e :

$$L_{j_l} = t_b + t_l - d_l \quad L_{j_e} = t_b + t_l + t_i + t_e - d_e \quad L_{j_e} \geq L_{j_l} \text{ since } t_i \geq 0 \wedge d_l < d_e \wedge t_e \geq 0$$

If we switch j_l and j_e :

$$L'_{j_l} = t_b + t_e + t_i - d_l \quad L'_{j_e} = t_b + t_l - d_e \text{ both are smaller than or equal to } L_{j_e} \text{ since } d_l > d_e \wedge t_i \geq 0 \wedge t_e \geq 0.$$

Switching a job with an earlier deadline and a job with later deadline such that the job with the earlier deadline is done first can never deteriorate a solution.

Therefore, scheduling the job with the earliest deadline first is both an optimal and a greedy solution. 90

(99)

BILKENT UNIVERSITY - Computer Engineering Department
CS 473/573 - Algorithms I Classwork 9

December 13, 2012

Duration: 120 min.

Name: Serhan ARSLAN

Registered Course/Section on SRS: ☐ CS473/1 ☐ CS473/2 ☒ CS573/1 ☐ CS573/2

Student Number:

(40)

1) (40 pts) A server has n customers waiting to be served. The service time required by each customer is known in advance; it is t_i minutes for customer i . Let W_i be the waiting time of customer i , which is the sum of the serving times of all customers served before customer i . So if, for example, the customers are served in order of increasing i , then the i th customer has to wait $W_i = \sum_{j=1}^{i-1} t_j$ minutes. We wish to minimize the total waiting time of all customers, which is equal to $\sum_{i=1}^n W_i$. Our goal is to formulate a greedy algorithm to compute the optimal order in which to serve the customers.

a) (10 pts) Explicitly state the greedy choice property.

b) (30 pts) Give the pseudocode for a greedy algorithm to compute the optimal order.

(1)

i	t_1	t_2	t_3	$\dots$	t_n
W_i	0	t_1	t_1+t_2	$t_1+t_2+t_3$	$\sum_{j=1}^{n-1} t_j$

* Assumed that all customers are available at the time we compute an optimal order.

a) Let $t_1 \leq t_2 \leq t_3 \leq \dots \leq t_n$ be the optimal order. (T_i is sorted in non-decreasing order). Then the total waiting time W_T is as follows:

$$W_T = \sum_{i=1}^n \sum_{j=1}^{i-1} t_j$$

If we assume that W_T^* is another optimal waiting time where $t_i \leq t_j$ but $i > j$ (now two items break the order), then the total waiting time W_T^* becomes greater than W_T since the waiting time of the customers after i th customer increases. So W_T^* cannot be an optimal choice. Therefore, if we give service to customers in non-decreasing t_i order, we obtain minimum (total) waiting time.

We can simply sort t_i s or apply the following greedy algorithm (at page 3)

2) (60 pts) In this problem, N units of a resource are to be allocated to R projects. If n units of resource are allocated to project r , then the resulting profit is $p_{r,n} > 0$, for $r = 1, 2, \dots, R$ and $n = 1, 2, \dots, N$. Note that allocating zero units to a project results in zero profit, i.e., $p_{r,0} = 0$ for $r = 1, 2, \dots, R$. An allocation which maximizes the total profit is called an optimal allocation. Our goal is to formulate an $O(N^2R)$ -time algorithm which computes and returns the profit of an optimal allocation. Assume that allocation profits are given in a 2 dimensional array P of size $R \times N$, such that $P[r,n] = p_{r,n}$ for $r = 1, 2, \dots, R$ and $n = 1, 2, \dots, N$.

For example, consider the profit table given below for $R = 3$ and $N = 3$. According to this table, the total profit of an optimal allocation is 10. This is obtained by assigning 1 unit of resource to project 1 (with profit $P[1,1] = 4$), 1 unit of resource to project 2 (with profit $P[2,1] = 1$), and 1 unit of resource to project 3 (with profit $P[3,1] = 5$).

P	0	1	2	N=3
1	0	4	2	6
2	0	1	3	4
3	0	5	5	2

	0	1	2	3
0	0	0	0	0
1	0	4	7	6
2	0	5	8	4
3	0	10	9	2

Clearly show the second and third steps for your dynamic programming formulation:

Step 2) (25 pts) Recursively define the profit of an optimal allocation.

Step 3) (35 pts) Give the pseudocode for an $O(N^2R)$ -time algorithm which computes and returns the profit of an optimal allocation.

Note that you do not have to construct an optimal solution.

① b) ORDER(T, n)
 S keeps the optimal order
 $S[1] \leftarrow T[1]$
 for $i \leftarrow 2$ to n
 if $T[i] > S[i-1]$
 $S[i] \leftarrow T[i]$
 else
 $k \leftarrow i-1$
 while ($T[i] < S[k]$ AND $k \geq 1$)
 $S[k+1] \leftarrow S[k]$
 $k \leftarrow k-1$
 $S[k] \leftarrow T[i]$
 RETURN S

2) For this question I keep two tables, PROFIT and REMAINING. Profit table stores the maximum profit that can be obtained so far. Remaining table stores the remaining resource so far. I calculate my tables in row order.

Let's P_{ij} be the profit of allocating project i, j resource. If $Rem[i, j]$ does not exceed R (total resource) at the time we allocate j to i , we consider it as a candidate allocation. Then we choose the maximum of candidates and update $Profit[i, j]$. The recurrence is as follows:

$$Profit[i, j] = \begin{cases} 0 & \text{if } j=0 \\ \max \left\{ P[i, j], \max_{k=1 \rightarrow N} \left\{ Profit[i-1, k] + P[i, j] \right\} \right\} & \text{if } REM[i-1, k] - k \geq 0 \end{cases}$$

if $i, j > 0$

FIND-OPTIMAL (N, R)

REM is an $R+1 \times N$ matrix ($0 \dots R, 0 \dots N$)

Profit is an $R+1 \times N$ matrix ($0 \dots R, 0 \dots N$)

for $i \leftarrow 0$ to R
for $j \leftarrow 0$ to N
profit[i, j] = 0;

for $j \leftarrow 0$ to N
REM[1, j] = N

for $i \leftarrow 1$ to R
REM[i, N] = 0

initialize

MAX = 0;

for $i \leftarrow 1$ to R

for $j \leftarrow 1$ to N

for $k \leftarrow 1$ to N

if $REM[i-1, k] - k \geq 0$

if $Profit[i-1, k] + P[i, j] > MAX$

MAX = $Profit[i-1, k] + P[i, j]$

REM[i, k] = $REM[i-1, k] - k$

PROFIT[i, j] = MAX

if $P[i, j] > MAX$

MAX = $P[i, j]$

REM[i, j] = $N - j$

PROFIT[i, j] = MAX

(RETURN MAX)

$O(N^2R)$

BILKENT UNIVERSITY - Computer Engineering Department
CS 473/573 - Algorithms I Classwork 10

December 20, 2012

Duration: 90 min.

Name: Solim ARSLAN

Registered Course/Section on SRS: ☐ CS473/1 ☐ CS473/2 ☒ CS573/1 ☐ CS573/2

Student Number: 21000084

For each greedy algorithm, first describe the greedy choice property and then present your algorithm in pseudo-code.

1) (40 pts) Consider the problem of summing n positive floating point numbers, $x_1, x_2, \dots, x_n$ in such a way that the rounding error is minimized. Depending on how the summation is done, the number of summations that numbers participate in may differ. For example, x_1, x_2, x_3 , and x_5 will participate in 3 summations, while x_4 will participate in 1 summation in the following summation order:

$$(((x_1 + x_3) + (x_2 + x_5)) + x_4)$$

Note that both the order of the numbers and the placement of parentheses are chosen freely, however, each summation includes only two numbers, that is, you cannot sum three or more numbers at once.

How large the rounding error of a sum of two floating point numbers is depends on how large the sum itself is (that is, a larger sum gives a larger error). Let p_i be the "parenthesis-depth" of floating point number x_i (that is, the number of parentheses surrounding the number). Then, the rounding error is defined as follows:

$$p_1 x_1 + p_2 x_2 + \dots + p_n x_n.$$

For example, consider the numbers $x_1 = 4.3$, $x_2 = 2.8$, $x_3 = 1.7$, $x_4 = 5.9$, and $x_5 = 3.4$. The summation order $((x_2 + x_5) + x_1) + (x_4 + x_3)$ gives the following rounding error:

$$2x_1 + 3x_2 + 2x_3 + 2x_4 + 3x_5 = 8.6 + 8.4 + 3.4 + 11.8 + 10.2 = 42.4.$$

Describe a greedy algorithm which computes the summation order of n given positive floating point numbers with minimum rounding error.

In this question we should first sort the numbers in ascending (non-decreasing) order. Then, at each iteration, we should take the numbers with the smallest values (the items at the first and second index of the array) and add them to the summation. By doing so we can minimize the rounding error, since the smallest floating point number will have larger parenthesis-depth. Here we can keep a min-heap like the Huffman-coding example. After selecting the smallest number we sum them and add the summation to the heap again. At the end we get the summation order with min rounding err.

MINIMUM-ROUNDING-ERROR

for $i \leftarrow 1$ to $n-1$

do allocate z to store a summation

→ $first \leftarrow \text{EXTRACT-MIN}(\text{MINX})$

$[left]z \leftarrow \text{EXTRACT-MIN}(\text{ORDER}) \triangleright x_{first}$

→ $second \leftarrow \text{EXTRACT-MIN}(\text{MINX})$

$[right]z \leftarrow \text{EXTRACT-MIN}(\text{ORDER}) \triangleright x_{second}$

→ $sum \leftarrow first + right$

$z \leftarrow "([left]z + [right]z)"$

→ $\text{INSERT}(\text{MINX}, sum)$

$\text{INSERT}(\text{ORDER}, z)$

return $\text{EXTRACT-MIN}(\text{ORDER})$

$\triangleright$ MINX is the min-heap that keeps $x_1, x_2, \dots, x_n$

$\triangleright$ order keeps the track of the floating numbers. It is a Heap structure and runs simultaneously with MINX. So at the end its root corresponds to the summation order with minimum error.

$\triangleright$ At first ORDER keeps $x_1, x_2, x_3, \dots, x_n$ in the same place as MINX keeps the value of $x_1, x_2, x_3, \dots, x_n$.

$\triangleright$ MINX keeps the values while order keeps the summations $(x_1 + x_2), ((x_1 + x_2) + x_3), \dots$

40

2(b) STORE-PILLS-2

$\triangleright$ U keeps unit cost

for $i \leftarrow 1$ to n

do $U[i] \leftarrow C[i] / P[i]$

$U \leftarrow \text{SORT}(U)$ $\triangleright$ U is now sorted in non-dec. order

$cost \leftarrow 0$

$\triangleright$ From now on I also assume that P and C is also sorted according to U.

$j \leftarrow 1$

$b \leftarrow P[j]$

for $i \leftarrow 1$ to W

do $b \leftarrow b+1$

if (b is full)

then $P[j] \leftarrow b$

$cost \leftarrow cost + C[j]$

$j \leftarrow j+1$

$b \leftarrow P[j]$

if (b is not full)

then $cost \leftarrow cost + C[j] / \text{the non-filled volume of } P[j]$

return cost

30

60

2) (60 pts) A pharmacist has W pills and n empty bottles. Assume that all pills are identical. Let $\{p_1, p_2, \dots, p_n\}$ denote the number of pills each bottle can hold.

(a) (30 pts) Describe a greedy algorithm, which takes W and the p_i values as input and determines the fewest number of bottles needed to store all the pills.

(b) (30 pts) How would you modify your algorithm if each bottle i also has an associated cost c_i , and you want to minimize the total cost of the bottles used to store all the pills? Assume that you pay only for the fraction of the bottle used. For example, if the i 'th bottle is half filled, you pay only $c_i/2$.

(a) Here we should sort the p_i values in decreasing order. By doing so, we can first store bigger bottles which yields the fewest possible number of bottles. ✓

STORE-PILLS

$\text{SORTED-P} \leftarrow \text{REVERSE-SORT}(P)$

$j \leftarrow 1$

$b \leftarrow \text{SORTED-P}[j]$

for $i \leftarrow 1$ to W

do $b \leftarrow b - 1$

if (b is full)

then $\text{SORTED-P}[j] \leftarrow b$

$j \leftarrow j + 1$

$b \leftarrow \text{SORTED-P}[j]$

return j ▷ number of bottles filled

▷ SORTED-P is the array that holds $\{p_1, p_2, p_3, \dots, p_n\}$ in decreasing order.

30

(b) This time we have also costs. In order to fill all the pills in minimum cost we find the unit cost for each p_i , via dividing c_i with p_i (c_i/p_i). Then we sort these unit-costs in non-decreasing order and start to fill the bottles according to this order. (1)

The algorithm is in the previous page. ✓